COPYRIGHT REGISTRATION – COMPUTER SOFTWARE

Title of Work: LinnkedOut
Author(s): Dhrish Parekh, Shiven Royal, Farhan Inamdar
Language: TypeScript (Node.js, React, Next.js)

Description:
A web-based application built with Next.js, featuring user authentication, routing, telemetry, and modern web application functionalities.

Declaration:
This is original work created by the author(s).

File: src/app/(HOME)/categories/[category]/page.tsx

```
"use server";

import { Media, PaginationInfo } from "@/services/common/types";
import { MEDIA_PER_PAGE } from "@/services/common/constants";
import Home from "@/app/_components/shared/Home";
import { getCategoryCount, getMediaByCategory } from "@/server/functions/media";
import { auth } from "@clerk/nextjs/server";

export default async function Category({
  searchParams,
  params,
}: {
  searchParams: Promise<{
    page: string;
  }>;
  params: Promise<{
    category: string;
  }>;
}) {
  const { userId } = await auth();
  if (!userId) return <div>Not logged in</div>;

  // Destructuring the slugs or the dynamic params
  const { category } = await params;
  const decodedCategory = decodeURIComponent(category);

  // Destructuring the page info to render the correct page
  const { page } = await searchParams;
  const currentPage: number = page ? parseInt(page) : 1;
  const offset = MEDIA_PER_PAGE * (currentPage - 1);

  // Fetching category data from the database
  const media: Media[] = await getMediaByCategory(
    decodedCategory,
    offset,
    userId
  );
  const totalMedia: number = await getCategoryCount(decodedCategory, userId);
  const totalPages = Math.ceil(totalMedia / MEDIA_PER_PAGE);
  console.log({ decodedCategory, currentPage, totalMedia });

  const paginationInfo: PaginationInfo = {
    currentPage,
    totalPages,
  };

  return (
    <Home
      media={media}
      pagination={paginationInfo}
      pageHeader={`Results for ${decodedCategory} category`}
```

```
    />
  );
}
```

File: src/app/(HOME)/layout.tsx

```
"use client";
import { ReactNode } from "react";
import { usePathname } from "next/navigation";
import AppSidebar from "../_components/shared/Sidebar";
import Navbar from "../_components/shared/Navbar";
import VideoNavBar from "../_components/shared/VideoNavbar";
import "../globals.css";

export default function Layout({ children }: { children: ReactNode }) {
  const pathname = usePathname();

  return (
    <main className="flex min-h-screen w-full flex-col fixed">
      {pathname.startsWith("/video") ? <VideoNavBar /> : <Navbar />}
      <div className={`flex flex-1 overflow-hidden`}>
        <AppSidebar />
        {children}
      </div>
    </main>
  );
}
```

File: src/app/(HOME)/navs/[nav]/page.tsx

```
import { Dashboard } from "@/app/_components/shared/Dashboard";
import { PostInputForm } from "@/app/_components/shared/PostInputForm";
import React from "react";

type ComponentMapType = {
    [ navType:string ]: React.ReactElement
}

const page = async (
    { params }: { params: Promise<{ nav: string }> }
) => {
    const { nav } = await params;

    const componentMap:ComponentMapType = {
        insert: <PostInputForm />,
        dashboard: <Dashboard />,
    };

    return (
        <div className="w-screen bg-dark scrollbar-hide overflow-y-auto max-h-[calc(100vh-
2rem)]">
```

```
            { componentMap[nav] }
        </div>
    )
}
export default page;
```

File: src/app/(HOME)/page.tsx

```
"use server";
import { Media } from "@/services/common/types";
import Home from "../_components/shared/Home";
import { getAllMedia, getMediaCount } from "@/server/functions/media";
import { MEDIA_PER_PAGE } from "@/services/common/constants";
import { auth } from "@clerk/nextjs/server";

export default async function HomePage({
  searchParams,
}: {
  searchParams: Promise<{ page: string | undefined }>;
}) {
  const { page } = await searchParams;

  const { userId } = await auth();
  if (!userId) return <div>Not logged in</div>;

  const currentPage = page ? parseInt(page) : 1;
  const totalMedia = await getMediaCount(userId);
  const totalPages = Math.ceil(totalMedia / MEDIA_PER_PAGE);
  const offset = MEDIA_PER_PAGE * (currentPage - 1);
  const paginationInfo = {
    currentPage,
    totalPages,
  };

  const media: Media[] = await getAllMedia(offset, userId);
  return <Home media={media} pagination={paginationInfo} />;
}
```

File: src/app/(HOME)/video/layout.tsx

```
"use client";
import { ReactNode } from "react";
import AppSidebar from "@/app/_components/shared/Sidebar";
import "@/app/globals.css";

export default function Layout({ children }: { children: ReactNode }) {
  return (
    <main className="flex min-h-screen w-full flex-col fixed">
      <div className="flex flex-1 overflow-hidden">
        <AppSidebar />
        {children}
```

```
      </div>
    </main>
  );
}
```

File: src/app/(HOME)/video/reddit/[platformId]/[videoId]/page.tsx

```
"use client";
import { use, useEffect, useState, useRef } from "react";
import { useSidebarState } from "../../../../../../hooks/useSideBarState";
import SideVideo from "@/app/_components/shared/SideVideo";
import { Media, RedditMedia } from "@/services/common/types";
import { usePlayingState } from "../../../../../../hooks/useIsPlaying";
import Loading from "@/app/_components/shared/Loading";
import CommentsDisplay from "@/app/_components/comments/CommentsDisplay";
import Link from "next/link";
import Image from "next/image";
import { RedditCommentComponent } from "@/app/_components/comments/RedditCommentComponent";

export default function Reddit({
  params,
}: {
  params: Promise<{
    platformId: number;
    videoId: number;
  }>;
}) {
  // Here i am getting generalized mediaId!!
  const mediaId = use(params).videoId;
  // Here i am getting redditId!!
  const redditId = use(params).platformId;

  const [video, setVideo] = useState<Media | null>(null);
  const [redditVideo, setRedditVideo] = useState<RedditMedia | null>(null);
  const [isHidden, setIsHidden] = useState<boolean>(false);
  const isOpen = useSidebarState((state) => state.isOpen);
  const isPlaying = usePlayingState((state) => state.isPlaying);
  const setIsPlaying = usePlayingState((state) => state.setIsPlaying);
  const videoRef = useRef<HTMLVideoElement>(null);

  // This is for the media
  useEffect(() => {
    const fetchDataById = async (mediaId: number) => {
      try {
        if (isOpen) {
          useSidebarState.getState().setIsOpen(false);
        }
        const res = await fetch(`/api/videos/${mediaId}`);
        if (!res.ok) {
          throw new Error(`Error: ${res.statusText}`);
        }
        const data = await res.json();
        setVideo(data.body);
```

```
    } catch (error: unknown) {
      console.error("Error fetching media:", error);
    }
  };
  fetchDataById(mediaId);
}, [mediaId]);

// This is for the reddit media
useEffect(() => {
  const fetchDataById = async (redditId: number) => {
    try {
      const res = await fetch(`/api/videos/media/reddit/${redditId}`);
      if (!res.ok) {
        throw new Error(`Error: ${res.statusText}`);
      }
      const data = await res.json();
      setRedditVideo(data.body);
    } catch (error: unknown) {
      console.error("Error fetching media:", error);
    }
  };
  fetchDataById(redditId);
}, [redditId]);

const handleVideoEnded = () => {
  setIsPlaying(false);
};

const handleVideoStateChange = () => {
  if (videoRef.current) {
    if (videoRef.current.paused) {
      setIsPlaying(false);
    } else {
      setIsPlaying(true);
    }
  }
};

const handleVideoClick = (e: React.MouseEvent<HTMLVideoElement>) => {
  e.preventDefault();
  if (videoRef.current) {
    if (videoRef.current.paused) {
      videoRef.current.play();
      setIsPlaying(true);
    } else {
      videoRef.current.pause();
      setIsPlaying(false);
    }
  }
};

const handleHidden = () => {
  setIsHidden(!isHidden);
};
```

```
  return (
    <div
      className={`overflow-y-auto pb-10 bg-dark flex flex-row h-screen w-screen
smallScreenPlatform
    ${
      isPlaying
        ? "bg-darker transition-all duration-500"
        : "transition-all duration-500"
    }
  `}
    >
      {/* First half components */}
      <div
        className="w-full lg:w-[70%] h-[calc(100vh-48px)] overflow-y-auto scrollbar-hide bg-
dark"
        id="For left Handed Components"
      >
        {/* The video Frame */}
        <div
          className={`mt-5 ml-2 rounded-xl z-40 w-full lg:w-[95%] h-[60%] overflow-hidden
relative transition-all duration-500
     `}
        >
          {video?.postUrl ? (
            <video
              ref={videoRef}
              src={video.postUrl}
              className="w-full h-full object-contain"
              controls
              onPlay={handleVideoStateChange}
              onPause={handleVideoStateChange}
              onEnded={handleVideoEnded}
              onClick={handleVideoClick}
            />
          ) : video?.thumbnailUrl ? (
            <div className="relative w-full h-full">
              <Image
                src={video.thumbnailUrl}
                alt={video.title || "Video thumbnail"}
                fill
                sizes="(max-width: 768px) 100vw, (max-width: 1200px) 70vw, 60vw"
                priority
                className="object-contain"
              />
              <div className="absolute inset-0 flex items-center justify-center">
                <div className="bg-black bg-opacity-50 p-2 rounded text-white">
                  Video unavailable
                </div>
              </div>
            </div>
          ) : (
            <Loading />
          )}
        </div>
```

```
        {/* The video metadata */}
        <div className="mt-6 space-y-2 relative sm:w-[60%] lg:w-[95%] ml-2">
          <div className="flex justify-between">
            <p className="text-white mb-2 text-xl">{video?.title}</p>
            <div className="text-white mb-2">
              {redditVideo?.postLink && (
                <Link href={redditVideo.postLink}>
                  <div
                    className={`w-32 h-8 flex bg-dark-golden border border-white rounded-xl
hover:bg-golden
                    hover-thread
                  `}
                  >
                    <p
                      className={`text-black flex items-center font-serif ml-9`}
                    >
                      Thread
                    </p>
                  </div>
                </Link>
              )}
            </div>
          </div>

          <div className="flex">
            <span className="text-gray-400 text-xl">
              {redditVideo?.subreddit && redditVideo.subreddit}
              {redditVideo?.author && redditVideo.subreddit && " | "}
              {redditVideo?.author && redditVideo.author}
            </span>
          </div>

          {/* For Comment Section */}
          <RedditCommentComponent redditVideo={redditVideo!} />
        </div>
      </div>

      {/* Second half components */}
      <div
        className="smallSecondHalfComponents scrollbar-hide overflow-y-auto bg-dark"
        id="For right Handed Components"
      >
        {/* <SideChat /> */}
        <SideVideo />
      </div>
    </div>
  );
}
```

File: src/app/(HOME)/video/reddit/page.tsx

```
"use server";
import { Media, PaginationInfo } from "@/services/common/types";
```

```
import { MEDIA_PER_PAGE } from "@/services/common/constants";
import {
  getRedditMediaCount,
  getAllMediaWherePlatformReddit,
} from "@/server/functions/media";
import Home from "@/app/_components/shared/Home";
import { auth } from "@clerk/nextjs/server";
import { redirect } from "next/navigation";

export default async function Reddit({
  searchParams,
}: {
  searchParams: Promise<{ page: string | undefined }>;
}) {
  const { page } = await searchParams;

  const { userId } = await auth();
  if (!userId) {
    redirect("/sign-in");
  }

  const currentPage = page ? parseInt(page) : 1;
  const offset = MEDIA_PER_PAGE * (currentPage - 1);

  const media: Media[] = await getAllMediaWherePlatformReddit(offset, userId);
  const totalMedia: number = await getRedditMediaCount(userId);
  const totalPages = Math.ceil(totalMedia / MEDIA_PER_PAGE);

  console.log(totalMedia);
  const paginationInfo: PaginationInfo = {
    currentPage,
    totalPages,
  };

  return (
    <Home
      media={media}
      pagination={paginationInfo}
      pageHeader="Result for Reddit Media"
    />
  );
}
```

File: src/app/(HOME)/video/youtube/[platformId]/[videoId]/page.tsx

```
"use client";
import { use, useEffect, useState, useRef } from "react";
import { useSidebarState } from "../../../../../../hooks/useSideBarState";
import SideVideo from "@/app/_components/shared/SideVideo";
import { CaptionItem, Media, YoutubeMedia } from "@/services/common/types";
import { usePlayingState } from "../../../../../../hooks/useIsPlaying";
import Loading from "@/app/_components/shared/Loading";
import Image from "next/image";
```

```
import { YoutubeMetadataDisplay } from "@/app/_components/shared/YoutubeMetadataDisplay";

declare global {
  interface Window {
    YT: {
      Player: any;
      PlayerState: {
        PLAYING: number;
        PAUSED: number;
        ENDED: number;
      };
    };
    onYouTubeIframeAPIReady: () => void;
  }
}

export const Page = ({
  params,
}: {
  params: Promise<{
    platformId: number;
    videoId: number;
  }>;
}) => {
  // Here i am getting mediaId!!
  const mediaId = use(params).videoId;

  // Here i am getting youtubeId!!
  const youtubeId = use(params).platformId;

  console.log(`PARAMS: ${use(params).platformId}`);

  const [video, setVideo] = useState<Media | null>(null);
  const [youtubeVideo, setYoutubeVideo] = useState<YoutubeMedia>();
  const isOpen = useSidebarState((state) => state.isOpen);
  const isPlaying = usePlayingState((state) => state.isPlaying);
  const setIsPlaying = usePlayingState((state) => state.setIsPlaying);
  const playerRef = useRef<any>(null);
  const [isHidden, setIsHidden] = useState<boolean>(true);
  const [isSummarizing, setIsSummarizing] = useState<boolean>(false);
  const [summary, setSummary] = useState<string>("");
  const [isSummaryOpen, setIsSummaryOpen] = useState(false);
  const [inputQuery, setInputQuery] = useState<string>("");

  // This is for the media
  useEffect(() => {
    const fetchDataById = async (mediaId: number) => {
      try {
        if (isOpen) {
          useSidebarState.getState().setIsOpen(false);
        }
        const res = await fetch(`/api/videos/${mediaId}`);

        if (!res.ok) {
          throw new Error(`Error: ${res.statusText}`);
```

```
      }

      const data = await res.json();
      console.log("Video API response:", data);

      if (data && data.body) {
        setVideo(data.body);
      } else {
        throw new Error("No video data found");
      }
    } catch (error: any) {
      console.error("Error fetching video:", error);
    }
  };

  if (mediaId) {
    fetchDataById(mediaId);
  }
}, [mediaId]);

useEffect(() => {
  const fetchYoutubeData = async (youtubeId: number) => {
    try {
      const res = await fetch(`/api/videos/media/youtube/${youtubeId}`);

      if (!res.ok) {
        throw new Error(`Error: ${res.statusText}`);
      }

      const data = await res.json();
      if (data && data.body) {
        setYoutubeVideo(data.body);
        console.log(`youtubeVideo: ${JSON.stringify(youtubeVideo)}`);
      }
    } catch (error: any) {
      console.error("Error fetching YouTube data:", error);
    }
  };

  fetchYoutubeData(youtubeId);
  console.log(`YT: ${youtubeId}`); // youtube
}, [youtubeId]);

// This is for creating iframe for youtube
useEffect(() => {
  if (!video?.postId) return;

  if (!window.YT) {
    const tag = document.createElement("script");
    tag.src = "https://www.youtube.com/iframe_api";
    const firstScriptTag = document.getElementsByTagName("script")[0];
    firstScriptTag.parentNode?.insertBefore(tag, firstScriptTag);
  }

  const onYouTubeIframeAPIReady = () => {
```

```
    playerRef.current = new window.YT.Player("youtube-player", {
      videoId: video.postId,
      events: {
        onStateChange: onPlayerStateChange,
        onReady: onPlayerReady,
        onError: onPlayerError,
      },
    });
  };

  if (window.YT && window.YT.Player) {
    onYouTubeIframeAPIReady();
  } else {
    window.onYouTubeIframeAPIReady = onYouTubeIframeAPIReady;
  }

  return () => {
    if (playerRef.current) {
      playerRef.current.destroy();
    }
  };
}, [video?.postId]);

const onPlayerReady = () => {
  console.log("Player ready");
};

const onPlayerError = (event: any) => {
  console.error("Player error:", event);
};

const onPlayerStateChange = (event: { data: number }) => {
  if (event.data === window.YT.PlayerState.PLAYING) {
    setIsPlaying(true);
  } else if (
    event.data === window.YT.PlayerState.PAUSED ||
    event.data === window.YT.PlayerState.ENDED
  ) {
    setIsPlaying(false);
  }
};

const handleHidden = () => {
  setIsHidden((prev) => !prev);
};

const extractCaptions = (captions: CaptionItem[]): string => {
  return captions.map((captObj) => captObj.text).join(" ");
};

const handleSummarizeClick = async () => {
  try {
    setIsSummarizing(true);
    setSummary("");
```

```
    const captions: CaptionItem[] | undefined = youtubeVideo?.englishCaptions;
    if (
      !captions ||
      captions === undefined ||
      captions === null ||
      captions.length <= 0
    ) {
      setSummary("captions are not present");
      return;
    }

    const captionsbody: string = extractCaptions(captions);
    const CaptionsData = {
      captionbody: captionsbody,
    };

    // Making request to the summarize endpoint
    const response = await fetch(`/api/summarize`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(CaptionsData),
    });

    if (!response.ok || !response.body) {
      throw new Error(`Error: ${response.status} ${response.statusText}`);
    }

    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let accSummary = "";

    while (true) {
      const { done, value } = await reader.read();
      if (done) {
        break;
      }

      const chunk = decoder.decode(value, { stream: true });
      accSummary += chunk;
      setSummary(accSummary);
    }
  } catch (error) {
    console.error(`Error Summarizing video`, error);
    setSummary("Failed to generate summary.");
  } finally {
    setIsSummarizing(false);
  }
};

const handleuserInputQuery = async () => {};

const closeSummary = () => {
  setIsSummaryOpen(false);
```

```
  };

  return (
    <div
      className={`overflow-y-auto pb-10 bg-dark flex flex-row smallScreenPlatform w-screen h-
screen
    ${
      isPlaying
        ? "bg-darker transition-all duration-500"
        : "transition-all duration-500"
    }
  `}
    >
      {/* The First half of the section */}
      <div
        className="ml-10 smallFirstHalfComponents w-full lg:w-[70%] h-[calc(100vh-48px)]
overflow-y-auto scrollbar-hide bg-dark"
        id="For left Handed Components"
      >
        {/* Youtube Player */}
        <div
          className={`mt-5 smallYoutubeVideoContainer rounded-xl w-full lg:w-[95%] transition-
all duration-500`}
        >
          <div className="relative w-full pb-[56.25%]">
            {video?.postId ? (
              <div
                id="youtube-player"
                className="absolute top-0 left-0 w-full h-full"
              />
            ) : (
              <Loading />
            )}
          </div>
        </div>

        {/* Metadata*/}
        <div className="sm:w-[60%] lg:w-[95%] block">
          <YoutubeMetadataDisplay video={video} />

          {youtubeVideo?.description && (
            <div className="flex justify-between smallDescription">
              <div className={`mt-4 text-white max-h-40 w-full`}>
                <div className="flex gap-4 h-8">
                  <h3 className="text-xl text-gray-400">Description</h3>
                  <Image
                    src={`${isHidden ? "/right.svg" : "/down.svg"}`}
                    alt=""
                    width={5}
                    height={5}
                    className={`bg-white rounded-full h-5 w-5 cursor-pointer mt-1`}
                    onClick={handleHidden}
                  />
                </div>
                <p
```

```
                  className={`text-gray-300 mt-2 bg-dark p-2 rounded-xl border-l-2 border-l-
golden shadow-golden shadow-xl
                        ${isHidden ? "hidden" : "whitespace-pre-wrap"} `}
                >
                  {youtubeVideo.description}
                </p>
              </div>
            </div>
          )}
        </div>
      </div>

      {/* second half of the screen*/}
      <div
        className="w-[40%] smallSecondHalfComponents scrollbar-hide overflow-y-auto"
        id="For right Handed Components"
      >
        {/* <SideChat /> */}
        <SideVideo />
      </div>
    </div>
  );
};

export default Page;
```

File: src/app/(HOME)/video/youtube/page.tsx

```
import Home from "@/app/_components/shared/Home";
import {
  getYoutubeMediaCount,
  getAllMediaWherePlatformYoutube,
} from "@/server/functions/media";
import { MEDIA_PER_PAGE } from "@/services/common/constants";
import { Media, PaginationInfo } from "@/services/common/types";
import { auth } from "@clerk/nextjs/server";
import { redirect } from "next/navigation";

export default async function Youtube({
  searchParams,
}: {
  searchParams: Promise<{ page: string | undefined }>;
}) {
  const { page } = await searchParams;

  const { userId } = await auth();
  if (!userId) {
    redirect("/sign-in");
  }

  const currentPage = page ? parseInt(page) : 1;
  const offset = MEDIA_PER_PAGE * (currentPage - 1);
```

```
  const media: Media[] = await getAllMediaWherePlatformYoutube(offset, userId);
  const totalMedia: number = await getYoutubeMediaCount(userId);
  const totalPages = Math.ceil(totalMedia / MEDIA_PER_PAGE);

  const paginationInfo: PaginationInfo = {
    currentPage,
    totalPages,
  };

  return (
    <Home
      media={media}
      pagination={paginationInfo}
      pageHeader="Result for YouTube Media"
    />
  );
}
```

File: src/app/(auth)/layout.tsx

```
export default function AuthLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="flex items-center justify-center h-screen w-screen">
        {children}
        </div>
  );
}
```

File: src/app/(auth)/sign-in/[[...sign-in]]/page.tsx

```
import { SignIn } from "@clerk/nextjs";

export default function Page() {
  return <SignIn />;
}
```

File: src/app/(auth)/sign-up/[[...sign-up]]/page.tsx

```
import { SignUp } from '@clerk/nextjs'

export default function Page() {
  return <SignUp  />
}
```

```
...
[CODE OMITTED FOR BREVITY - MIDDLE SECTION OF REPOSITORY]
...
```

```
// ... continuing ...

      // 2. Fallback method: Third-party service (youtube-transcript.io)
      console.log(`[YoutubeTranscriptService] [${videoId}] youtube-transcript failed, falling
back to Third-party service...`);
      transcript = await this.fetchTranscriptViaThirdParty(videoId);
      if (transcript && transcript.length > 0) {
        console.log(`[YoutubeTranscriptService] [${videoId}] Successfully fetched
${transcript.length} transcripts via Third-party service.`);
        return transcript;
      }

      /*
      // 3. Disabled method: yt-dlp (Most reliable local method)
      console.log(`[YoutubeTranscriptService] [${videoId}] Attempting fetch via yt-dlp...`);
      transcript = await this.fetchTranscriptViaYtDlp(videoId);
      if (transcript && transcript.length > 0) {
        console.log(`[YoutubeTranscriptService] [${videoId}] Successfully fetched
${transcript.length} transcripts via yt-dlp.`);
        return transcript;
      }
      */

      return [];
    } catch (error) {
      if (error instanceof Error && error.message.includes('Transcript is disabled')) {
        console.error('[YoutubeTranscriptService] Transcripts are disabled for this video');
        return [];
      }
      console.error('[YoutubeTranscriptService] Error during transcript fetch:', error);
      return [];
    }
  }

  private async fetchTranscriptViaThirdParty(videoId: string): Promise<CaptionItem[]> {
    try {
      const token = process.env.TRANSCRIPT_SERVICE_TOKEN;
      if (!token) {
        console.warn('[YoutubeTranscriptService] No TRANSCRIPT_SERVICE_TOKEN found in
environment');
        return [];
      }

      const response = await fetch("https://www.youtube-transcript.io/api/transcripts", {
        method: "POST",
        headers: {
          "Authorization": `Basic ${token}`,
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
```

```
          ids: [videoId],
        })
      });

      if (!response.ok) {
        console.warn(`[YoutubeTranscriptService] [${videoId}] Third-party service request
failed: ${response.status}`);
        return [];
      }

      const data = await response.json();

      // The service typically returns { "videoId": [segments] } or similar
      const rawSegments = data[videoId] || (data.results && data.results[videoId]) || [];

      if (!Array.isArray(rawSegments) || rawSegments.length === 0) {
        console.warn(`[YoutubeTranscriptService] [${videoId}] No transcript segments found in
third-party response`);
        return [];
      }

      return rawSegments.map((seg: { text?: string; content?: string; lang?: string;
language?: string; offset?: number; start?: number; duration?: number; end?: number }) => ({
        text: seg.text || seg.content || '',
        lang: seg.lang || seg.language || 'en',
        offset: seg.offset || seg.start || 0,
        duration: seg.duration || (seg.end !== undefined && seg.start !== undefined ? seg.end
- seg.start : 0)
      }));

    } catch (error) {
      console.error(`[YoutubeTranscriptService] [${videoId}] Error fetching from third-party
service:`, error);
      return [];
    }
  }

  /*
  private async fetchTranscriptViaYtDlp(videoId: string): Promise<CaptionItem[]> {
    await this.ensureYtdlp();

    if (!this.ytdlp) {
      console.warn(`[YtDlp] [${videoId}] yt-dlp not initialized`);
      return [];
    }

    try {
      const VIDEO_URL = `https://www.youtube.com/watch?v=${videoId}`;
      const SUBTITLE_LANG = 'en';
      const cookiesPath = '/app/youtube-cookies.txt';

      const args = [
        '--dump-json',
        '--no-warnings',
      ];
```

```
      if (fs.existsSync(cookiesPath)) {
        console.log(`[YtDlp] [${videoId}] Using cookies from ${cookiesPath}`);
        args.push('--cookies', cookiesPath);
      } else {
        console.warn(`[YtDlp] [${videoId}] No cookies file found at ${cookiesPath}`);
      }

      args.push(VIDEO_URL);

      console.log(`[YtDlp] [${videoId}] Executing with args:`, JSON.stringify(args, null, 2));

      const metadataStr = await this.ytdlp.execPromise(args);
      const metadata = JSON.parse(metadataStr) as YtDlpMetadata;

      const subtitleInfo = metadata.subtitles?.[SUBTITLE_LANG] ||
metadata.automatic_captions?.[SUBTITLE_LANG];

      if (!subtitleInfo) {
        console.warn(`[YtDlp] [${videoId}] No English subtitles or automatic captions found`);
        return [];
      }

      const json3Subtitle = subtitleInfo.find((sub: YtDlpSubtitle) => sub.ext === 'json3');

      if (!json3Subtitle || !json3Subtitle.url) {
        console.warn(`[YtDlp] [${videoId}] No json3 format found. Available:
${subtitleInfo.map((s: YtDlpSubtitle) => s.ext).join(', ')}`);
        return [];
      }

      const transcriptUrl = json3Subtitle.url;
      const rawResponse = await this.downloadFromUrl(transcriptUrl);

      if (rawResponse.trim().startsWith('<html>')) {
        console.warn(`[YtDlp] [${videoId}] Received HTML instead of JSON`);
        return [];
      }

      const json3Data = JSON.parse(rawResponse) as Json3Data;
      const parsedTranscripts = this.parseJson3Transcript(json3Data, SUBTITLE_LANG);
      console.log(`[YtDlp] [${videoId}] ✓ Parsed ${parsedTranscripts.length} segments`);
      return parsedTranscripts;

    } catch (error) {
      console.error(`[YtDlp] [${videoId}] Method failed:`, error);
      return [];
    }
  }
  */



  /*
  private downloadFromUrl(url: string): Promise<string> {
```

```
    return new Promise((resolve, reject) => {
      const options = {
        headers: {
          'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/120.0.0.0 Safari/537.36'
        }
      };
      https.get(url, options, (response) => {
        let data = '';
        response.on('data', (chunk) => { data += chunk; });
        response.on('end', () => { resolve(data); });
      }).on('error', (error) => { reject(error); });
    });
  }

  private parseJson3Transcript(json3Data: Json3Data, lang: string): CaptionItem[] {
    const transcriptItems: CaptionItem[] = [];
    if (json3Data.events) {
      for (const event of json3Data.events) {
        if (event.segs) {
          const text = event.segs
            .map((seg: Json3Seg) => seg.utf8 || '')
            .join('')
            .trim();

          if (text) {
            transcriptItems.push({
              text: text,
              lang: lang,
              offset: (event.tStartMs || 0) / 1000,
              duration: (event.dDurationMs || 0) / 1000
            });
          }
        }
      }
    }
    return transcriptItems;
  }
  */

  public async generateTranscriptsFromAudio(videoId:string, title:string):
Promise<CaptionItem[]>{
    const tempFiles: string[] = [];
    try{
      let audioFilePath: string | null = null;
      let transcriptFilePath: string | null = null;

      console.log(`[Whisper] Downloading Audio for ${videoId}...`);
      audioFilePath = await this.fetchAndSaveAudioFromPlatform(videoId, title);
      tempFiles.push(audioFilePath);
      console.log(`[Whisper] Audio saved: ${path.basename(audioFilePath)}`);

      console.log(`[Whisper] Extracting Transcripts using Whisper...`);
      transcriptFilePath = await this.ExtractTranscriptsFromAudio(audioFilePath, title);
      tempFiles.push(transcriptFilePath);
```

```
      console.log(`[Whisper] Transcripts generated: ${path.basename(transcriptFilePath)}`);

      console.log('[Whisper] Reading transcript file...')
      const transcripts:CaptionItem[] = await
this.ReadTranscriptsJsonFile(transcriptFilePath);
      console.log(`[Whisper] Found ${transcripts.length} transcript segments`);

      return transcripts;
    } catch(error){
      console.error('[Whisper] Error during transcript generation:', error);
      throw error;
    } finally {
      await this.cleanUpTemporaryFiles(tempFiles);
    }
  }

  public extractEnglishCaptions(transcriptItems: TranscriptResponse[]): CaptionItem[] {
    try {
      const enTranscripts: TranscriptResponse[] = transcriptItems.filter((item) =>
        item.lang !== undefined && (item.lang === 'en' || item.lang === 'English' ||
item.lang.startsWith('en')));

      if (enTranscripts.length === 0) {
        return transcriptItems.map(item => ({
            text: item.text,
            lang: item.lang || 'unknown',
            offset: item.offset,
            duration: item.duration
        }));
      }

      const formattedTranscripts: CaptionItem[] = enTranscripts.map((item) => ({
        text: item.text,
        lang: item.lang || 'en',
        offset: item.offset,
        duration: item.duration
      }));

      return formattedTranscripts;
    } catch (error) {
      console.error('Error processing transcripts:', error);
      throw error;
    }
  }

  public async fetchAndSaveAudioFromPlatform(videoId: string, title: string): Promise<string>
{
    const extension = `wav`;
    return new Promise((resolve, reject) => {
      if (!fs.existsSync(Audio_Output_Directory)) {
        try {
            fs.mkdirSync(Audio_Output_Directory, { recursive: true });
        } catch {
            // Fallback to a temp directory if the constant one fails (e.g. wrong path)
            const fallbackDir = path.join(process.cwd(), 'temp_audio');
```

```
            if (!fs.existsSync(fallbackDir)) fs.mkdirSync(fallbackDir, { recursive: true });
            const p = path.join(fallbackDir, `${title}.${extension}`);
            this.runYtDlpAudio(videoId, p, extension, resolve, reject).catch(reject);
            return;
        }
      }

      const audioPath = path.join(Audio_Output_Directory, `${title}.${extension}`);
      this.runYtDlpAudio(videoId, audioPath, extension, resolve, reject).catch(reject);
    });
  }

  private async runYtDlpAudio(videoId: string, audioPath: string, extension: string, resolve:
(path: string) => void, reject: (err: Error) => void) {
    await this.ensureYtdlp();
    if (!this.ytdlp || !this.isYtdlpReady) {
      return reject(new Error('yt-dlp is not ready'));
    }

    try {
      this.ytdlp.exec([
        `https://www.youtube.com/watch?v=${videoId}`,
        '-f', 'bestaudio',
        '--extract-audio',
        '--audio-format', extension,
        '-o', audioPath,
        '--extractor-args', 'youtube:player_client=android',
        '--extractor-args', 'youtube:player_skip=webpage,configs,js'
      ])
      .on('error', (err) => {
        console.error('[YoutubeTranscriptService] yt-dlp error:', err);
        reject(err);
      })
      .on('close', (code) => {
        if (code === 0) {
          resolve(audioPath);
        } else {
          reject(new Error(`yt-dlp exited with code ${code}`));
        }
      });
    } catch (err) {
      reject(err instanceof Error ? err : new Error(String(err)));
    }
  }

  public async ExtractTranscriptsFromAudio(absoluteAudioFilePath: string, title: string):
Promise<string> {
    return new Promise((resolve, reject) => {
      if (!absoluteAudioFilePath || absoluteAudioFilePath.length <= 0) {
        return reject(new Error(`Audio path is not valid: ${absoluteAudioFilePath}`));
      }

      const whisper = spawn('whisper', [
        absoluteAudioFilePath,
        '--language', 'English',
```

```
        '--output_format', 'json',
        '--output_dir', path.dirname(absoluteAudioFilePath)
      ]);

      whisper.stdout.on('data', () => {
        // console.log(`stdout: ${logs.toString()}`);
      });

      whisper.stderr.on('data', () => {
        // console.error(`stderr: ${errorLogs.toString()}`);
      });

      whisper.on('close', (code) => {
        if (code === 0) {
          const audioFileName = path.basename(absoluteAudioFilePath,
path.extname(absoluteAudioFilePath));
          const TranscriptFilePath = path.join(path.dirname(absoluteAudioFilePath),
`${audioFileName}.json`);

          if (fs.existsSync(TranscriptFilePath)) {
            resolve(TranscriptFilePath);
          } else {
            const alternativeFilePath = path.join(path.dirname(absoluteAudioFilePath),
`${title}.json`);
            if (fs.existsSync(alternativeFilePath)) {
              resolve(alternativeFilePath);
            } else {
              reject(new Error(`Transcript file not found. Expected: ${TranscriptFilePath} or
${alternativeFilePath}`));
            }
          }
        } else {
          reject(new Error(`whisper exited with code ${code}`));
        }
      });

      whisper.on('error', (error) => {
        reject(new Error(`Failed to start whisper process: ${error.message}`));
      });
    });
  }

  public async cleanUpTemporaryFiles(tempfiles: string[]):Promise<void> {
    const filesToCleanup:string[] = tempfiles.filter(Boolean)

    filesToCleanup.map((filePath:string) => {
      try{
        if(fs.existsSync(filePath)){
          this.DeleteExistingFile(filePath);
          console.log(`cleaned up: ${path.basename(filePath)}`)
        }
      } catch(error){
        console.warn(`⚠️ Could not delete temp file ${filePath}:`, error);
      }
    })
```

```
  }

  public async DeleteExistingFile(filePath: string):Promise<void> {
    if (!filePath || filePath.length <= 0 || filePath === undefined) {
      throw new Error(`Audio path is not valid: ${filePath}`);
    }
    console.log(`Deleting the ${path.basename(filePath)} file...`)
    fs.rmSync(filePath);
    console.log(`${filePath} file deleted successfully!`);
  }

  public async ReadTranscriptsJsonFile(JsonFilePath: string): Promise<CaptionItem[]> {
    try{
      const JsonFileData = await fs.promises.readFile(JsonFilePath, {
        encoding: 'utf-8',
        flag: 'r',
      })
      const JsonFile = JSON.parse(JsonFileData);
      return this.parseJsonTranscripts(JsonFile);
    } catch(error){
      throw new Error(`Unable to read the file: ${error}`);
    }
  }

  public parseJsonTranscripts(jsonFile: WhisperJsonFile): CaptionItem[] {
    const { language, segments } = jsonFile;
    const parsedTranscript: CaptionItem[] = segments.map((segment: WhisperSegment) => {
      const { text, start, end } = segment;
      const transcript: CaptionItem = {
        text: text,
        duration: end - start,
        offset: start,
        lang: language
      }
      return transcript;
    })
    return parsedTranscript;
  }

  public extractCaptionText(captions: CaptionItem[]): string {
    return captions.map(caption => caption.text).join(' ');
  }

}
```

File: src/services/vector/CacheService.ts

```
import path from 'path'
import fs from 'fs/promises'

export class CacheService {

    private cachePath:string;
```

```
  constructor(){
      this.cachePath = path.join(process.cwd(), '.cache', 'category-embeddings.json');
  }

  async cacheEmbeddings(embeddings: Record<string, number[]>): Promise<void> {
  try {
    // For server-side Next.js
    const cacheDir = path.join(process.cwd(), '.cache');
    await fs.mkdir(cacheDir, { recursive: true });

    const cachePath = path.join(cacheDir, 'category-embeddings.json');
    await fs.writeFile(cachePath, JSON.stringify(embeddings));

    console.log("Category embeddings cached successfully");
  } catch (error) {
    console.error("Failed to cache embeddings:", error);
  }
}

async getCachedCategoryEmbeddings(): Promise<Record<string, number[]>> {
  try {
    const filePath = this.cachePath;
    try {
      const data = await fs.readFile(filePath, 'utf-8');
      const parsed:any = JSON.parse(data);

      if(parsed && typeof parsed === 'object' && !Array.isArray(parsed)){
        const categoryCount = Object.keys(parsed).length;
        console.log(`Found ${categoryCount} categories in cache`);
        return parsed as Record<string,number[]>;
      } else{
        console.warn('Cache exists but does not contain valid embeddings data')
        return {};
      }
    } catch (error) {
      console.error('Cache File not present or cant be read')
      return {};
    }
  } catch (error) {
    console.error("Failed to retrieve cached embeddings:", error);
    return {};
  }
}

  async checkCacheExists(): Promise<boolean> {
      try{
        const cacheFilePath = this.cachePath;
        const stats = await fs.stat(cacheFilePath);
        return stats.size > 0;
      } catch(error){
        console.error(error)
        return false
      }
```

```
    }
}
```

File: src/services/vector/EmbeddingService.ts

```
import { VoyageEmbeddings } from "@langchain/community/embeddings/voyage";
import { store } from "@/state/store";
import { setEmbeddings } from "@/state/Slice/EmbeddingSlice";
import { categoryDefinitions } from "@/services/common/constants";
import { CacheService } from "./CacheService";

export class EmbeddingService {
  private voyageEmbeddings: VoyageEmbeddings;
  private cacheService: CacheService;

  constructor() {
    this.voyageEmbeddings = new VoyageEmbeddings({
      apiKey: process.env.VOYAGE_API_KEY,
      batchSize: 20,
      maxRetries: 3,
      modelName: 'voyage-3.5-lite',
    });
    this.cacheService = new CacheService();
  }

  async initializeEmbeddings(categories: Record<string, string>): Promise<Record<string,
number[]>> {
    try {
      // check for cached embeddings
      const cachedEmbeddings = await this.cacheService.getCachedCategoryEmbeddings();

      // if cache found then use it from centralised zustand store
      if (cachedEmbeddings && Object.keys(cachedEmbeddings).length ===
Object.keys(categories).length) {
        console.log(`Using cached embeddings for ${Object.keys(cachedEmbeddings).length}
categories`);
        store.dispatch(setEmbeddings(cachedEmbeddings))
        return cachedEmbeddings;
      } else {
        // or else generate the embeddings and then store it in the cache
        console.log(`Generating new embeddings for ${Object.keys(categories).length}
categories`);
        const embeddings = await this.generateCategoryEmbeddings()
        store.dispatch(setEmbeddings(embeddings))
        await this.cacheService.cacheEmbeddings(embeddings)
        console.log(`Initialized Embeddings successfully!`);
        return embeddings;
      }
    } catch (error) {
      console.error("Error initializing embeddings:", error);
      throw error;
    }
  }
```

```
  async generateEmbeddings(content: string): Promise<number[]> {
    try {
      if (!this.voyageEmbeddings) throw new Error("voyageEmbeddings is not initialized");
      if (!content || typeof content !== "string" || content.trim() === "") throw new
Error("Content is invalid or empty");

      const embedding = await this.voyageEmbeddings.embedQuery(content);
      if (!embedding || !Array.isArray(embedding) || embedding.length === 0) throw new
Error("Generated embedding is invalid");

      return embedding;
    } catch (error) {
      if (error instanceof Error) {
        console.error(`Error in generateEmbeddings: ${error.message}`);
        throw error;
      }
      throw error;
    }
  }

  async generateBatchEmbeddings(contents: string[]): Promise<number[][]> {
    try {
      if (!contents || contents.length === 0) throw new Error(`The content is not valid`);

      if (!this.voyageEmbeddings) {
        throw new Error("voyageEmbeddings is not initialized");
      }

      const validContents = contents.filter(content =>
        content && typeof content === 'string' && content.trim().length > 0
      );

      if (validContents.length === 0) {
        throw new Error("No valid content to embed");
      }

      console.log(`Embedding ${validContents.length} valid documents out of ${contents.length}
total`);
      console.log("First content sample:", validContents[0]?.substring(0, 100) + "...");

      const result = await this.voyageEmbeddings.embedDocuments(validContents);

      console.log("Raw result type:", typeof result);
      console.log("Raw result is array:", Array.isArray(result));
      console.log("Result length:", result?.length);
      if (result && Array.isArray(result) && result.length > 0) {
        console.log("First embedding type:", typeof result[0]);
        console.log("First embedding is array:", Array.isArray(result[0]));
        console.log("First embedding length:", result[0]?.length);
      }

      if (!result) {
        throw new Error("Voyage embedding API returned null/undefined result");
      }
```

```
      if (!Array.isArray(result)) {
        throw new Error(`Expected array result, got ${typeof result}:
${JSON.stringify(result)}`);
      }

      if (result.length === 0) {
        throw new Error("Voyage embedding API returned empty array");
      }

      for (let i = 0; i < result.length; i++) {
        if (!Array.isArray(result[i])) {
          throw new Error(`Embedding at index ${i} is not an array: ${typeof result[i]}`);
        }
        if (result[i].length === 0) {
          throw new Error(`Embedding at index ${i} is empty array`);
        }
      }

      console.log(`Successfully generated ${result.length} embeddings`);
      return result;

    } catch (error: any) {
      // Enhanced error handling
      console.error("Batch Embedding Generation Error Details:");
      console.error("Error type:", typeof error);
      console.error("Error message:", error?.message);
      console.error("Error stack:", error?.stack);

      if (error.response?.status === 429) {
        console.warn("Rate limit hit. Implementing retry logic...");
        throw new Error("Rate limit exceeded. Please try again later.");
      }

      if (error.message?.includes('API key')) {
        throw new Error("Invalid or missing Voyage API key");
      }

      throw error;
    }
  }


  async generateCategoryEmbeddings(): Promise<Record<string, number[]>> {
    const categories: Record<string, string> = categoryDefinitions;

    if (!categories || Object.keys(categories).length === 0) {
      console.warn("No categories provided to generate embeddings for");
      return {};
    }

    // const cacheExists:boolean = await this.checkCacheExists();

    // // check if the cache is valid if not then change it to the the below implementation
    // if(cacheExists){
```

```
    // const cachedEmbeddings:Record<string,number[]> = await
this.getCachedCategoryEmbeddings();
    // console.info(`Fetched categories embeddings from the cache`)

    // const allCategoriesPresent = Object.keys(categories).every(cat =>
    //   cachedEmbeddings.hasOwnProperty(cat) &&
    //   Array.isArray(cachedEmbeddings[cat]) &&
    //   cachedEmbeddings[cat].length > 0
    // );

    // if (allCategoriesPresent) {
    //   console.log(`All ${Object.keys(cachedEmbeddings).length} categories found in cache`);
    //   return cachedEmbeddings;
    // }
    // console.info("Cache missing some categories, generating new embeddings");
    // }

    // Fetching keys and values from the categories object
    const categoryEmbeddings: Record<string, number[]> = {};
    const categoryEntries = Object.entries(categories);

    // Generating the category embeddings
    try {
      console.info(`Generating batch embeddings for ${categoryEntries.length} categories`);

      // Prepare descriptions for batch embedding
      const descriptions = categoryEntries.map(([_, desc]) => desc);
      const categoryNames = categoryEntries.map(([name, _]) => name);

      const embeddings: number[][] = await this.generateBatchEmbeddings(descriptions);

      categoryNames.forEach((name, i) => {
        categoryEmbeddings[name] = embeddings[i];
        console.log(`Generated embedding for category: ${name}`);
      });

      console.log(`Successfully generated embeddings for all ${categoryNames.length}
categories`);
    } catch (error) {
      console.error("Failed to generate batch embeddings:", error);
      throw error;
    }

    // caching the newly generated embeddings
    if (Object.keys(categoryEmbeddings).length > 0) {
      await this.cacheService.cacheEmbeddings(categoryEmbeddings);
    }

    return categoryEmbeddings;
  }

}
```

File: src/services/vector/PreprocessingService.ts

```
import { Media, YoutubeMetadata } from "../common/types";
import { RedditMetadataSevice } from "@/services/platform/reddit/RedditMetadataService";
import { YoutubeTranscriptService } from "../platform/youtube/YoutubeTranscriptionService";
import natural from "natural";
import stopwords from "stopwords-iso";

export class ProcessingService {
  private youtubeTranscriptionService: YoutubeTranscriptService;
  private redditMetaDataService: RedditMetadataSevice;

  constructor() {
    this.youtubeTranscriptionService = new YoutubeTranscriptService();
    this.redditMetaDataService = new RedditMetadataSevice();
  }

  public extractAndPreprocessData(
    mediaData: Media,
    platformData: any,
    youtubeMetadata?: YoutubeMetadata
  ): string {
    let textSources = [];

    try {
      if (mediaData?.title) {
        textSources.push(mediaData.title);
      }

      if (platformData?.description) {
        textSources.push(platformData.description);
      }

      if (platformData?.englishCaptions) {
        const extractedCaptions =
          this.youtubeTranscriptionService.extractEnglishCaptions(
            platformData.englishCaptions
          );
        textSources.push(extractedCaptions);
      }

      if (youtubeMetadata?.topicDetails.topicCategories) {
        for (const topicCategory in youtubeMetadata.topicDetails
          .topicCategories) {
          const cleanedTopicCategory = this.cleanText(topicCategory);
          textSources.push(cleanedTopicCategory);
        }
      }

      // This will be solved with redditOrchestrator
      if (platformData?.comments) {
        const extractedComments: string =
          this.redditMetaDataService.extractComments(platformData.comments);
```

```
        textSources.push(extractedComments);
      }

      let unifiedText: string = textSources.join(" ");
      unifiedText = this.cleanText(unifiedText);
      unifiedText = unifiedText.slice(0, 1000);

      if (
        !unifiedText ||
        typeof unifiedText !== "string" ||
        unifiedText === undefined ||
        unifiedText.length <= 0
      ) {
        throw new Error("Preprocessing failed: content is undefined or empty");
      }

      return unifiedText;
    } catch (error) {
      console.error(
        "PreprocessingService: Error while preprocessing content",
        error
      );
      throw error;
    }
  }

  public cleanText(text: string): string {
    let cleaned = text.toLowerCase().replace(/https?:\/\/\S+/g, "");

    // Tokenize
    const tokenizer = new natural.WordTokenizer();
    const tokens = tokenizer.tokenize(cleaned);

    // Removing stopwords, emoji, and special characters
    const combinedStopwords = [...stopwords.en, ...stopwords.hi];
    const filteredTokens = tokens.filter(
      (token) =>
        !combinedStopwords.includes(token) &&
        token.length > 1 &&
        !/[^\x00-\x7F]+/.test(token) &&
        !/^\d+$/.test(token)
    );

    return filteredTokens.join(" ");
  }
}
```

File: src/state/Slice/EmbeddingSlice.ts

```
import { createSlice, PayloadAction } from "@reduxjs/toolkit";
import { AppState, videoMetaData } from '@/services/common/types';

const initialAppState:AppState = {
```

```
    embeddings: {},
    watchHistory: []
}

const embeddingSlice = createSlice({
    name:'embeddingSlice',
    initialState:initialAppState,
    reducers:{
        addToWatchHistory: (state:AppState, action:PayloadAction<videoMetaData>) => {
            state.watchHistory.unshift(action.payload);
            state.watchHistory = state.watchHistory.slice(0,50);
            localStorage.setItem('watchHistory',JSON.stringify(state.watchHistory))
        },
        setEmbeddings: (state:AppState, action:PayloadAction<Record<string,number[]>>) => {
            state.embeddings = action.payload;
        }
    }
})

export const { addToWatchHistory, setEmbeddings} = embeddingSlice.actions
export default embeddingSlice.reducer;
```

File: src/state/store.ts

```
import { configureStore } from "@reduxjs/toolkit"
import EmbeddingReducer from "@/state/Slice/EmbeddingSlice"

export const store = configureStore({
    reducer:{
        appReducer: EmbeddingReducer
    }
})

export type RootState = ReturnType<typeof store.getState>;
export type Dispatch = typeof store.dispatch;
```